



Graph Contrastive Pre-training for Anti-money Laundering

Hanbin Lu¹ · Haosen Wang¹

Received: 17 August 2024 / Accepted: 2 December 2024
© The Author(s) 2024

Abstract

Anti-money laundering (AML) is vital to maintaining financial markets, social stability, and political authority. At present, many studies model the AML task as the graph and leverage graph neural network (GNN) for node/edge classification. Although these studies have achieved some achievements, they struggle with the issue of label scarcity in real-world scenarios. In this paper, we propose a graph contrastive pre-training framework for anti-money laundering (GCPAL), which mines supervised signals from the label-free transaction network to significantly reduce the dependence on annotations. Specifically, we construct three augmented views (i.e., two stochastic perturbed views and a KNN view). Perturbed views are beneficial to the model learning invariant information and improve the robustness against noise. KNN view provides implicit interactions to mitigate the link sparsity in the transaction network. Moreover, we extend the positive sample set using connected neighbors and node pairs with similar features to further enhance the expressiveness of the model. We evaluate the GCPAL on two datasets, and the extensive experimental results demonstrate that the GCPAL is consistently superior to other SOTA baselines, especially with scarce labels.

Keywords Anti-money laundering · Graph contrastive learning · Graph neural network

1 Introduction

Money laundering is the process of disguising, concealing, and transforming the illegal income obtained from crimes such as drug trafficking, prostitution, and smuggling. It has become one of the most prevalent economic crimes, seriously undermining economic stability and social security. According to SIPRI, global money laundering of up to 1.6 trillion dollars (2.7% of global GDP) in 2022 [1]. Therefore, combating money laundering is urgent for maintaining healthy economic development.

Early studies relied on expert knowledge, where financial institutions designed numerous indicators and rules to detect suspicious transactions. For example, large transactions occurring in high-risk regions at midnight may warrant attention. Although rule-based methods are intuitive, their labor-intensive nature makes it difficult to handle vast amounts of data. Additionally, pre-designed rules can be easily bypassed by emerging money laundering tactics. Consequently, many studies focus on applying machine learning

and deep learning to AML. Recently, graph neural networks (GNNs) have emerged as a promising solution for AML by leveraging the topological information in transaction networks.

Despite the remarkable success of GNN-based methods, they still face several challenges. First, labeled data are both costly and scarce. Most existing studies rely on supervised learning, which requires sufficient and high-quality annotations. However, in the field of AML, manually labeling suspicious transactions is expensive and labor-intensive. Therefore, it is essential to extract valuable information from unlabeled data. Second, interactions between transactions are often sparse. Due to the challenges of data acquisition, available data are often fragmented. Moreover, money launderers often obscure sequences of suspicious transactions, further complicating detection. However, overly sparse interactions can limit the performance of GNNs.

Contrastive learning (CL), a popular framework in self-supervised learning, has recently achieved significant success across various domains [2–7]. Many researchers have adapted CL to graph data, referred to as graph contrastive learning (GCL). The core idea of GCL is to capture intrinsic knowledge by maximizing the agreement between representations from different views of the same graph. Motivated

✉ Hanbin Lu
HanBinLu_SEU@163.com

¹ School of Cyber Science and Engineering, Southeast University, Nanjing 211102, Jiangsu, China

by this, we propose GCPAL, a novel graph contrastive pre-training framework for anti-money laundering (AML). Specifically, we construct three augmented views, including two stochastic perturbed views and a KNN view. The perturbed views are generated using edge dropping and feature dropping strategies, which help the model learn invariant information and improve robustness against noise. The KNN view is generated by selecting the top- k most similar node pairs based on the node attribute matrix, providing implicit interactions to address link sparsity in the transaction network and preserving node feature similarity [8]. These three views are then embedded by a shared graph encoder. Finally, the graph encoder is optimized through a cross-view contrastive learning objective across the three views. Based on the homophily assumption that connected and similar nodes tend to share the same label, we treat neighboring nodes in the original graph and KNN view as positive samples for target nodes.

Our contributions are summarized as follows:

- We propose the GCPAL framework, which achieves strong AML performance with limited labels by leveraging contrastive learning to enhance model expressiveness across multiple augmented views.
- We construct KNN views to mitigate the sparsity of interactions in the data. Moreover, the extended positive sample set further enhances the performance of the model.
- Extensive experiments demonstrate that GCPAL outperforms state-of-the-art (SOTA) AML models, especially with scarce labeled data (e.g., 1% and 2% of the training data).

2 Related Work

2.1 Anti-money Laundering

Early studies mainly are rule-based methods [9, 10], relying on expert knowledge and simple data analysis to identify suspicious transactions. For instance, Rajput et al. [10] developed an ontology-based expert system incorporating domain knowledge and explicit rules. Nonetheless, rule-based methods are challenging to keep up to date and cannot adapt to new money laundering strategies. More recently, researchers have increasingly applied machine learning algorithms to anti-money laundering (AML). These approaches can be broadly categorized into supervised methods, such as random forests (RF), logistic regression (LR), and support vector machines (SVM), and unsupervised methods, such as K-means and t-SNE. Jorge et al. [11] proposed a Bayesian network for AML, while Savage et al. [12] used RF and SVM to detect criminal transactions. However, studies have shown that machine

learning models often suffer from a high false-positive rate problem [13, 14].

Recently, an increasing number of studies have been devoted to deep learning methods [15] in AML. Paula et al. [16] designed an AutoEncoder to capture financial transaction patterns, while Han et al. [17] utilized LSTM to embed news articles and tweets to support AML investigations. Inspired by the success of GNNs in areas like social networks [18], recommendation systems [19], anomaly detection [20–22], and traffic prediction [23], several studies [24–26] have leveraged topological information in financial transactions to uncover potential money laundering patterns. Alarab et al. [24] proposed an early GNN-based model that combines GCN and MLP in parallel to classify Bitcoin transactions. Evolve-GCN [26] addresses both structural and temporal characteristics using GCN to learn structural information at each time step and RNN to integrate features across time steps. Subsequently, other studies [27–29] have proposed dynamic graph neural networks to model the evolving patterns in money laundering. To capture more complex relationships and rich semantic information, some researchers [30, 31] model financial interaction networks as heterogeneous graphs. Palita et al. [32] address the issue of class imbalance using focal loss as the optimization objective. Numerous advanced detection methods [33–37] have also been developed to accurately classify anomalous nodes in networks using deep learning techniques. However, these methods are typically semi-supervised or supervised, requiring substantial label data. Since labeling suspicious transactions is costly and labor-intensive, Inspection-L [38] proposes a self-supervised GNN framework using deep graph infomax (DGI). LaundroGraph [39] leverages a link prediction task on the directed bipartite customer-transaction graph to train GNN in a self-supervised manner.

2.2 Graph Contrastive Learning

Motivated by the promising performances of contrastive learning in the fields of CV and NLP, many studies pay attention to contrastive learning on graph data. Graph contrastive learning aims to maximize the mutual information (MI) between positive examples within graphs. DGI [40] is a pioneering approach that maximizes MI between node and graph representations. Other models, such as InfoGraph [41], MVGRL [42], and SUGAR [43], also leverage MI between local and global representations to learn node embeddings. GCC [44] constructs two augmented views by randomly perturbing nodes or edges and pre-trains GNNs by bringing together representations of the same node across these views while pushing apart representations of different nodes. GCA [45] proposes multiple adaptive graph augmentation strategies based on topological and semantic information. CuCo [46] introduces curriculum learning into contrastive learn-

ing by creating a scoring function to rank negative samples. BYOL [47] is a bootstrap method that does not use negative samples, significantly reducing memory costs. As a self-supervised paradigm, graph contrastive learning extracts supervisory signals from unlabeled data, greatly reducing dependency on labeled data. In this paper, we propose a graph contrastive learning framework for anti-money laundering.

3 Preliminaries

3.1 Problem Definition

Following [48], we construct the Bitcoin transaction network as the graph $G = (V, E, X)$, where node $v_i \in V$ denotes a transaction, edge $e_{i,j} \in E$ denotes the financial flow between nodes v_i and v_j , and $X = \{x_1, x_2, \dots, x_{|V|}\} \in \mathbb{R}^{|V| \times d}$ is the node feature matrix. Finally, the adjacency matrix is presented as $A \in 0, 1^{|V| \times |V|}$.

This paper considers anti-money laundering as a graph pre-training task. We pretrain a GNN $g : x_i \rightarrow h_i$, where h_i is the pre-trained representation of v_i . We then train a supervised classifier $f : h_i \rightarrow Y_i$, where Y_i is the label of v_i .

3.2 Graph Neural Networks

Graph neural networks (GNNs) aim to iteratively update node representation by combining representations from itself and its neighborhoods as

$$h_i^{(k)} = \text{COM} \left(h_i^{(k-1)}, \text{AGG} \left(\left\{ h_j^{(k-1)} : j \in \mathcal{N}_i \right\} \right) \right), \quad (1)$$

where $h_i^{(k)}$ is the node representation in the k th layer, $\mathcal{N}_i$ represents the set of directly connected neighbors of node i , and $h_i^{(0)} = x_i$. $\text{AGG}(\cdot)$ denotes the aggregate function which aggregates neighbor's information. $\text{COM}(\cdot)$ is the combination function that combines the aggregated neighbor information and its own features from the previous layer, such as averaging, summation, and concatenation.

4 Methodology

This section elaborates on the proposed graph contrastive pre-training framework for anti-money laundering (GCPAL). We begin with an overview of the GCPAL framework and then detail its two major parts: graph contrastive pre-training and supervised classification.

4.1 Overview of GCPAL

The proposed GCPAL model builds upon the graph contrastive learning method for AML and consists of two main stages: self-supervised pre-training and supervised classification. Figure 1 illustrates the main architecture of GCPAL. The pseudocode is shown in Algorithm 1.

The pre-training phase generates three augmented graph views: two stochastic perturbed views and a KNN view. The stochastic perturbed graphs are created through feature and edge dropping, while the KNN view is constructed by selecting the top- k most similar node pairs based on node features. These three views are encoded by a shared graph encoder to obtain their respective representations. Contrastive learning tasks are performed across these multiple views. Additionally, positive sample sets are created for each node. The objective of contrastive learning pre-training is to maximize the agreement between positive samples and the dissimilarity between negative samples.

The classification phase is conducted using the limited labels obtained through manual annotation. The pre-trained GNN encoder is reused to generate global-level node embeddings. These embeddings are then concatenated with raw features, and the resulting feature set is fed into a classifier for the anti-money laundering task. The goal of this phase is to accurately identify illegal transactions.

Algorithm 1: Training procedure of GCPAL

Input: Transaction graph $\mathcal{G} = (V, E, X)$; Graph augmenters {ED, FD, KNN}; Graph encoder $g(\cdot)$; MLP classifier; Pre-training batch size $\mathcal{B}$; Epoch number $\mathbb{E}$; Weight hyper-parameter λ

Output: Optimized GIN encoder and MLP classifier

```

1 Initialize the trainable parameters  $\theta$  and  $\omega$  for the GIN encoder and the MLP classifier;
2 # Self-supervised pre-training
3 for epoch  $\leftarrow 1$  to  $\mathbb{E}$  do
4   # Generate three augmented views
5    $\mathcal{G}', \mathcal{G}'', \mathcal{G}^{\text{KNN}} = \text{EF}(\text{ED}(\mathcal{G})), \text{EF}(\text{FD}(\mathcal{G})), \text{KNN}(\mathcal{G})$ 
6   # Obtain node representations by Eq. 5
7    $H', H'', H^{\text{KNN}} = g(\mathcal{G}'), g(\mathcal{G}''), g(\mathcal{G}^{\text{KNN}})$ 
8   for sampled node batch  $\mathcal{B}$  do
9     # Compute GCL loss by Eq. 10
10     $\mathcal{L}_{\text{pretrain}} = \lambda \mathcal{L}_{\text{GCL}}(\mathcal{G}', \mathcal{G}'') + (1 - \lambda) \mathcal{L}_{\text{GCL}}(\mathcal{G}'', \mathcal{G}^{\text{KNN}})$ 
11    Update  $\theta$  by minimizing  $\mathcal{L}_{\text{pretrain}}$ 
12  end
13 end
14 # Supervised classifier training
15 Select labeled node set  $V_{\text{label}}$  and the label  $Y$ 
16 Obtain node representations  $H$  of  $V_{\text{label}}$  by learned graph encoder  $g(\cdot)$  (E.q. 5)
17 Training MLP classifier by binary cross-entropy loss  $\mathcal{L}_{\text{AML}}$  (E.q. 13)
18 return  $g(\cdot)$ , MLP

```

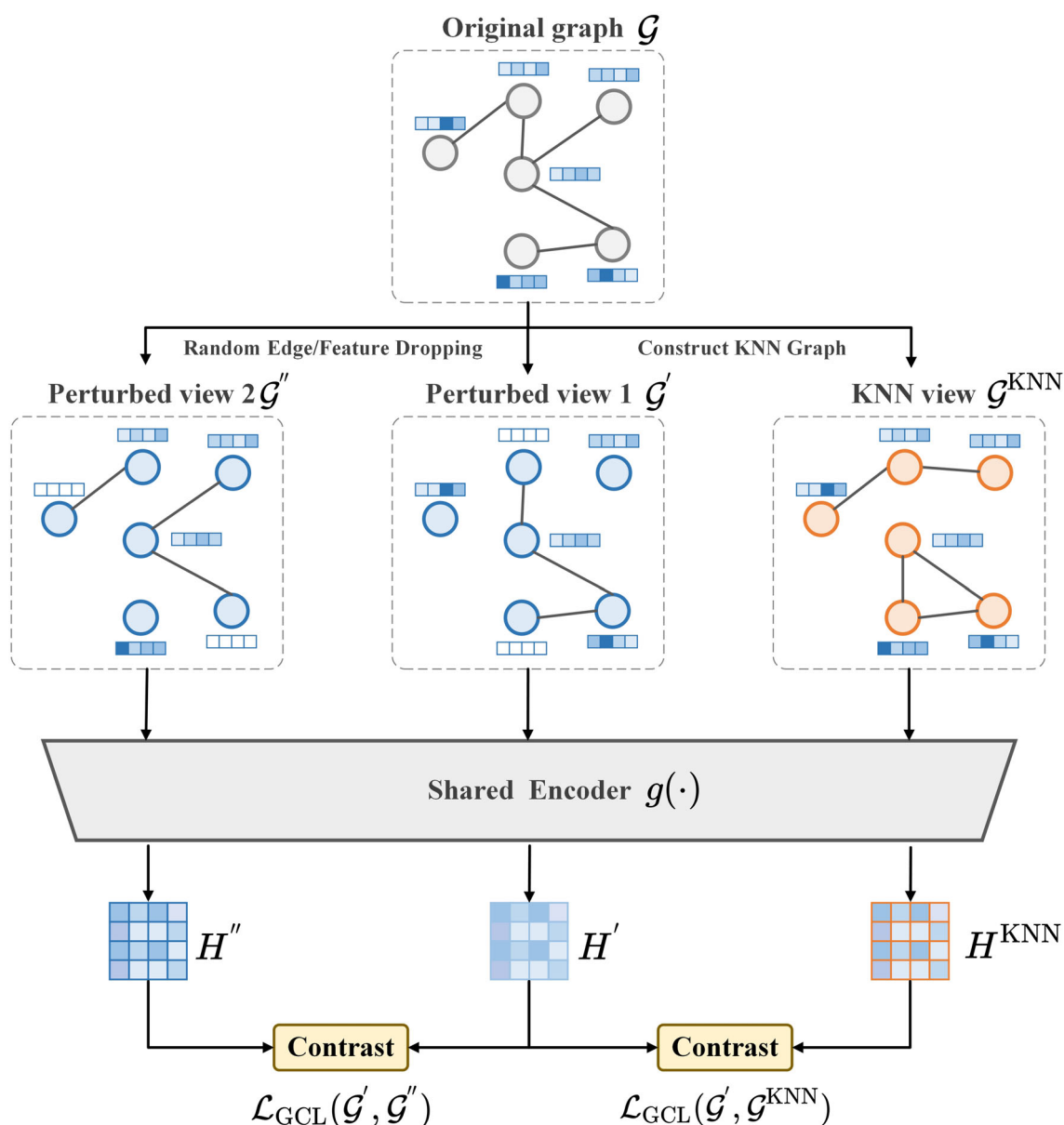


Fig. 1 Graph contrastive pre-training of GCPAL. Two perturbed views $\mathcal{G}'$ and $\mathcal{G}''$ are generated using graph augmentation from original graph $\mathcal{G}$. The KNN view is constructed by selecting the top- k most similar node pairs based on the node features. Three views are encoded by a

shared graph encoder to obtain their representations. The objective of contrastive learning pre-training is to maximize the agreement between positive samples and the dissimilarity between negative samples

4.2 Graph Contrastive Pre-training

The pre-training stage aims to extract inherent knowledge from massive unlabeled data. This phase typically includes three major components: graph data augmentation, graph encoder, and contrastive learning.

4.2.1 Graph Data Augmentation

Bitcoin transaction network has valuable characteristics in both its graph structure and node information. (1) The Bitcoin transaction network may have a large number of edges (e.g., 1.1B edges in the full Elliptic dataset) representing payment flows, the majority of which are legal. As a result, the graph may contain redundant edges that have limited relevance in detecting illegal transactions. (2) Nodes in the network contain high-dimensional features, such as time, transaction

fees, and several aggregated values. Although these features contribute to classification accuracy, they also increase the risk of information redundancy and over-fitting. (3) Certain similarities may exist among illicit transactions (e.g., similar payment methods or currency types). However, due to temporal and spatial constraints, some transactions may lack direct connections or have only a few multi-hop links within the network. Since GNNs can utilize only limited k-hop neighbor information, this semantic similarity of transaction nodes is not well exploited in AML. To address these challenges, we employ three augmentations to the original transaction graph to generate different views, including edge dropping, feature dropping, and KNN graph construction.

Edge dropping (ED) Given the edge set E , edge dropping will randomly remove certain ratio α of edges. It aims to reduce the model's reliance on whole edges to help reveal more meaningful structures. Formally, this process can be presented as

$$ED(\mathcal{G}) = (M_1 \odot E, X), \quad (2)$$

where $M_1 \in \{0, 1\}^{|E|}$ is an edge masking matrix where corresponding elements are 1 if the edge is masked; otherwise, it is 0. And, $\odot$ denotes element-wise product.

Feature dropping (FD) Given the node feature matrix X , feature dropping will randomly discard certain portion β of nodes. Similarly, this operation is used to reduce reliance on abundant features, improving the model's generalization and robustness. This procedure can be modeled as

$$FD(\mathcal{G}) = (E, M_2 \odot X), \quad (3)$$

where $M_2 \in \{0, 1\}^{|V| \times d}$ is a masking matrix of feature matrix X where all elements in the j th row are 0 if j th node is masked.

KNN graph construction To leverage the semantic similarity between different transactions, we propose to build a KNN graph view using raw node features. In detail, we first compute the similarity score by matrix multiplication of X . Then, we keep k edges with the highest similarities for each node to obtain the augmented adjacency matrix A' . The process can be formulated as

$$A^{\text{KNN}} = \text{top-}k \left(X X^T \right), \quad (4)$$

where $X \in \mathbb{R}^{|V| \times d}$ denotes the raw feature matrix, $\text{top-}k(\cdot)$ function is used to select the most similar node pairs. After that, we can obtain the new edge set E^{KNN} of KNN graph by extracting edges from the adjacency matrix A^{KNN} . The whole procedure can be defined as: $\text{KNN}(\mathcal{G}) = (E^{\text{KNN}}, X)$.

We generate two perturbed views $\mathcal{G}'$ and $\mathcal{G}''$ and a KNN view $\mathcal{G}^{\text{KNN}}$ through above strategies.

4.2.2 Graph Encoder

Graph encoding enables the transformation of complex structures into informative representations essential for classification. The graph encoder $g(\cdot)$ is flexible and can be selected from common graph neural networks (GNNs), such as GCN, GAT, and GIN, among others. In this work, the default graph encoder is the graph isomorphism network (GIN) [49], due to its exceptional ability in graph modeling. Formally, GIN is considered the most powerful GNN model theoretically, as it ensures the injectivity of the passing function. It can be represented as

$$h_i^{(k)} = \text{MLP}^{(k)} \left((1 + \varepsilon^{(k)}) h_i^{(k-1)} + \text{SUM} \left(\{ h_j^{(k-1)} : j \in \mathcal{N}_i \} \right) \right), \quad (5)$$

where $\text{MLP}(\cdot)$ is a multi-layer perceptron, $\varepsilon^{(k)}$ denotes a learnable scalar parameter, $h_i^{(k)} \in \mathbb{R}^d$ is the representation of node i in the k th GNN layer, and $h_i^{(0)} = x_i$ is the raw node feature. We can acquire the final representations of all nodes $H = \{h_1^{(k)}, h_2^{(k)}, \dots, h_{|V|}^{(k)}\}$ by stacking k layers of GNN. We will omit the superscript (k) below for simplicity.

We can obtain the representations of three augmented views by the shared graph encoder $g(\cdot)$ as follows:

$$H' = g(\mathcal{G}'), H'' = g(\mathcal{G}''), H^{\text{KNN}} = g(\mathcal{G}^{\text{KNN}}). \quad (6)$$

4.2.3 Contrastive Learning

GCL is a self-supervised graph learning technique that learns node representations by contrasting representations from different graph views. Before the contrastive learning task, a project layer $Z = \text{proj}(H)$ is further applied to project the learned node representations into a latent space, where a multi-layer perceptron (MLP) is used in this work. Finally, according to the principle of mutual information maximization (MIM), the objective of CL is to push the representations of positive samples close and pull negative samples away. This is typically accomplished using the InfoNCE [50] loss as a lower bound of MIM. The GCL loss between $\mathcal{G}'$ and $\mathcal{G}''$ can be defined as

$$\mathcal{L}_{\text{GCL}}(\mathcal{G}', \mathcal{G}'') = \frac{1}{2|\mathcal{B}|} \sum_{v \in \mathcal{B}} \mathcal{L}_{\text{MI}}(z'_v, z''_v) + \mathcal{L}_{\text{MI}}(z''_v, z'_v), \quad (7)$$

$$\mathcal{L}_{\text{MI}}(z'_i, z''_i) = \sum_{i \in \mathcal{B}} -\log \frac{\sum_{j \in \mathbb{P}_i} \exp(\text{sim}(z'_i, z''_j) / \tau)}{\sum_{k \in \{\mathbb{P}_i \cup \mathbb{N}_i\}} \exp(\text{sim}(z'_i, z''_k) / \tau)}, \quad (8)$$

where $\mathcal{B}$ denotes the node set in the current batch, τ is a temperature factor, z'_v and z''_v denote projected representations of node v in views $\mathcal{G}'$ and $\mathcal{G}''$, respectively, and sim denotes the cosine similarity here. $\mathbb{P}_i$ and $\mathbb{N}_i$ represent the positive samples and negative samples of node i , respectively.

Based on the homophily assumption, we regard both connected neighbors and neighbors with similar features as positive samples. Specifically, we define the positive sample matrix $M_{\mathbb{P}}$ as follows:

$$M_{\mathbb{P}} = A + A^{\text{KNN}}. \quad (9)$$

The non-zero elements in i -th row of $M_{\mathbb{P}}$ are positive samples $\mathbb{P}_i$ of node v_i .

Similarly, we can calculate the contrastive loss between $\mathcal{G}'$ and $\mathcal{G}^{\text{KNN}}$. The final loss of graph contrastive learning can be presented as

$$\mathcal{L}_{\text{pretrain}} = \lambda \mathcal{L}_{\text{GCL}}(\mathcal{G}', \mathcal{G}'') + (1 - \lambda) \mathcal{L}_{\text{GCL}}(\mathcal{G}', \mathcal{G}^{\text{KNN}}), \quad (10)$$

where λ is the hyper-parameter to control the weight of each loss.

Theoretical analysis. We will prove that the mutual information between the two views can be maximized by optimizing the loss presented in Eq. (8). In Ref. [51], the proof is shown as follows:

$$\begin{aligned} \mathcal{L}_{\text{MI}}(z'_i, z''_i) &= \sum_{i \in \mathcal{B}} -\log \frac{\sum_{j \in \mathbb{P}_i} \exp(\text{sim}(z'_i, z''_j)/\tau)}{\sum_{k \in \{\mathbb{P}_i \cup \mathbb{N}_i\}} \exp(\text{sim}(z'_i, z''_k)/\tau)}, \\ &= \mathbb{E}_Z \left[-\log \frac{\sum_{j \in \mathbb{P}_i} \exp(\text{sim}(z'_i, z''_j)/\tau)}{\sum_{k \in \{\mathbb{P}_i \cup \mathbb{N}_i\}} \exp(\text{sim}(z'_i, z''_k)/\tau)} \right] \\ &\geq \mathbb{E}_Z \left[-\log \frac{\exp(\text{sim}(z'_i, z''_i)/\tau)}{\exp(\text{sim}(z'_i, z''_i)/\tau) + \sum_{k \in \mathbb{N}_i} \exp(\text{sim}(z'_i, z''_k)/\tau)} \right] \\ &= \mathbb{E}_Z \left[-\log \frac{p(z''_i | z'_i)/p(z'_i)}{p(z''_i | z'_i)/p(z'_i) + \sum_{k \in \mathbb{N}_i} p(z''_i | z'_i)/p(z'_k)} \right] \\ &= \mathbb{E}_Z \log \left[1 + \frac{p(z'_i)}{p(z''_i | z'_i)} \sum_{k \in \mathbb{N}_i} \frac{p(z''_i | z'_i)}{p(z'_k)} \right] \\ &\approx \mathbb{E}_Z \log \left[1 + \frac{p(z'_i)}{p(z''_i | z'_i)} (|\mathcal{B}| - 1) \mathbb{E}_{z''_k} \frac{p(z''_i | z'_i)}{p(z'_k)} \right] \\ &= \mathbb{E}_Z \log \left[1 + \frac{p(z'_i)}{p(z''_i | z'_i)} (|\mathcal{B}| - 1) \right] \\ &\geq \mathbb{E}_Z \log \left[\frac{p(z'_i)}{p(z''_i | z'_i)} |\mathcal{B}| \right] \\ &= -I(z'_i, z''_i) + \log(|\mathcal{B}|). \end{aligned} \quad (11)$$

Therefore, $I(z'_i, z''_i) \geq \log(|\mathcal{B}|) - \mathcal{L}_{\text{MI}}(z'_i, z''_i)$, where $I(z'_i, z''_i)$ denotes the mutual information between two views. We proposed the optimization objective is the lower bound of mutual information maximization.

4.3 Supervised Classification

Once the graph contrastive pre-training is completed, we reuse the pre-trained graph encoder $g(\cdot)$ (without project head $\text{proj}(\cdot)$) to obtain node representations $H = \{h_1, h_2, \dots, h_{|V|}\}$. Afterward, H and the raw feature X will be concatenated as input features to a classifier to get the classification score. The whole process can be presented as

$$P = \sigma(\text{classifier}(H || X)), \quad (12)$$

where $P \in \mathbb{R}^{|V|}$ denotes the possibilities of illegal transactions, σ denotes the softmax function, and $||$ is the concatenation operation. With losing generality, $\text{classifier}(\cdot)$ can be an arbitrary supervised classification model. We use a multi-layer perceptron in this work due to its effectiveness and efficiency.

Finally, we use the binary cross-entropy loss to optimize the classifier, which can be presented as

$$\mathcal{L}_{\text{AML}} = -\frac{1}{|V_{\text{label}}|} \sum_{v \in V_{\text{label}}} Y_v \log(P_v) + (1 - Y_v) \log(1 - P_v), \quad (13)$$

where V_{label} denotes the trained node set, Y_v denotes the label of node v (v is illegal when $Y_v = 1$), and P_v is the predicted probability of node v as a illegal transaction.

5 Experiments

5.1 Experimental Settings

5.1.1 Dataset

We use two datasets to evaluate our proposed model. The Elliptic dataset [26] consists of 203,769 nodes (transactions) and 234,355 edges (transaction flows). Of these, 4545 (2%) transactions are labeled as illegal, 42,019 (21%) are labeled as legal, and the remaining transactions are unlabeled. Each transaction is represented by 166 features. The first 94 features capture characteristics of the transaction itself (e.g., time step, number of inputs/outputs, and transaction fee). The remaining 72 features are aggregated characteristics based on information from neighboring transactions, including the maximum, minimum, standard deviation, and correlation coefficients of similar data from neighboring transactions. We denote the local raw features (the first 94 features) and the full set of raw features (94 local features plus 72 aggregated features) as LF and AF, respectively.

AMLworld [52] is a recently emerged synthetic AML dataset generated by a simulator that builds a multi-agent

virtual world of banks, individuals, and companies. Nine different illegal activities, such as extortion, loan sharking, gambling, etc., can lead to placement in AMLworld. The amount of money obtained and how frequently these illegal operations occur depend on the activity and the performing entity. The financial system is then more entrenched with illicit monies. Two high-level groups, HI and LI, with correspondingly higher and lower illicit ratios (laundering), are derived from the data. Separate datasets for HI and LI are available in small, medium, and large sizes, with the large datasets containing between 175 M and 180 M transactions. We use the HI-Small dataset due to its appropriate size and high illicit ratio.

5.1.2 Evaluation Metrics

In real-world detection tasks, the metrics of F1-score, accuracy, recall, and precision provide valuable insights into different aspects of a model's effectiveness.

Accuracy reflects the overall proportion of correctly identified instances, but it can be misleading in class-imbalanced settings, such as anti-money laundering (AML), where true-positive cases are often rare.

Recall measures the model's ability to identify all true suspicious cases, which is particularly important in fields like disease diagnosis and cybersecurity, where capturing as many true positives as possible is crucial.

Precision indicates the model's ability to correctly identify positive cases among all those labeled as positive. It is especially critical in applications such as financial risk management and spam detection, where avoiding false warnings or misclassifications is paramount.

F1-score provides a balanced measure of precision and recall, capturing both the accuracy and sensitivity of the model. This makes it particularly well suited for evaluating AML models, where both false positives and false negatives have significant implications.

5.1.3 Baseline Methods

We conduct a comprehensive comparison of GCPAL with a variety of established baseline methods.

GNN-based methods:

- GCN [53] is a neural network architecture that performs convolutional operations on graph-structured data, using localized node features and graph topology to learn node embeddings.
- GAT [54] utilizes the attention mechanism to weigh the importance of different neighbors during message passing.
- GraphSAGE [55]: GraphSAGE is a scalable graph neural network framework that learns node embeddings by

utilizing neighborhood sampling and aggregation techniques. By generalizing over local structures, it generates node representations in large graphs efficiently.

- GIN [49] improves graph representation learning with a novel aggregation function that uses multi-layer non-linear layers, allowing for superior discrimination between non-isomorphic graphs.
- Skip-GCN [26] adds a skip connection between the intermediate feature embeddings and the raw features before the last GCN layer.
- Evolve-GCN [26] uses a separate GCN for each time step and connects them with an RNN to better capture system dynamics.
- OCGTL [56] is a graph-level anomaly detection model, which combines the advantages of deep one-class classification and self-neural transformation learning.

Non-graph-based methods:

- Logistic regression is a binary classification model based on linear combinations of input features.
- MLP conducts multi-layer non-linear transformation on the input features for classification.

Self-supervised GNN methods:

- Inspection-L [38] is a strong baseline method using Deep Graph Informax (DGI) pre-training and downstream supervised training for AML detection.
- GAE [57] is a graph encoder-decoder framework, which embeds nodes to low-dimensional representations and predicts inputted structures.
- GraphMAE [58] is a masked graph autoencoder that focuses on reconstructing the masked feature.

5.1.4 Implementation Details

We set the embedding size to 128 for baseline models. The batch size is searched from {64, 128, ..., 2028}. The edge and feature dropout ratios are both searched from {0.1, 0.3, ..., 0.9}. The weight λ is tuned from {0.1, 0.2, ..., 1.0}. The temperature factor τ is searched from {0.05, 0.1, 0.2, 0.5, ..., 5.0}. The neighbor number of the KNN graph is turned from {1, 3, 5, 10, ..., 50}. The data ratio of supervised training is set as {1%, 2%, 5%, 10%, 20%}. We run each method five times with different random seeds and report the average performance along with its standard deviation. The model training is stopped if there is no improvement in performance after 50 consecutive epochs (Tables 1, 2).

5.2 Performance Comparison

We compare GCPAL with SOTA AML detection models to show its effectiveness. The results are presented in Tables 3 and 4. The superscript LF or AF on models in Table 3 indicates whether local raw features or all raw features are used. We use bold fonts to highlight the best results. According to the results, we have the following observations:

For GNN-based methods, OCGTL demonstrates the best performance, especially when the training ratios are extremely small (e.g., 1% and 2% on Elliptic and 40% on AMLworld). As the training data ratio increases, the performance of OCGTL becomes even more pronounced. For instance, OCGTL achieves the highest F1 scores (e.g., 80.3%, 82.7%, and 84.6%) among GNN models under 5%, 10% and 20% training ratios on Elliptic.

For non-graph-based methods, MLP significantly outperforms logistic regression across all training ratios due to its superior ability to model non-linear relationships. Moreover, as the training ratio increases, the benefit of using aggregated features (AF) becomes more evident. For instance, the F1 scores of MLP^{AF} surpass those of MLP^{LF} only when the training ratio exceeds 2%. The reason might be that MLP struggles to effectively utilize the complex aggregated features at smaller training ratios, as MLP models typically require large amounts of data to learn effectively.

Among self-supervised GNN methods, GAE performs poorly, likely due to the sparse interactions in AML, which do not provide enough self-supervised signals. GraphMAE shows competitive performance, as the reconstruction of masked features provides intrinsic knowledge that helps reduce reliance on labels. Inspection-L achieves the second-best performance across all training ratios, only trailing behind GCPAL, outperforming other baseline models. Notably, it attains high F1 scores (76% and 77.9%) on the Elliptic dataset, even with training ratios as low as 1% and 2%. These results highlight the effectiveness of self-supervised pre-training, which uses self-supervised signals to guide model optimization, leading to strong generalization capabilities with limited training data.

Finally, our proposed GCPAL significantly outperforms all other methods across all training ratios. For instance, GCPAL achieves the highest F1 scores of 78% and 87.3% at the lowest (1%) and highest (20%) training ratios, respectively. The performance improvement stems from the graph contrastive learning pre-training, which enables the model to learn more fine-grained node representations. Compared to Inspection-L, our method introduces a greater number of negative samples during self-supervised training, which enhances the discriminative power of the learned representations, leading to superior performance.

5.3 Ablation and Variant Study

5.3.1 Ablation Study

Our proposed GCPAL model incorporates several essential components, including GCL between random graph views, GCL between random and KNN views, and the connected neighbor-based positive sample selection. To assess the contribution of each component, we perform an ablation study with the following variants:

- w/o randGCL: This variant removes the GCL between random graph views from the entire GCPAL model. Specifically, the loss term $\lambda_{\text{GCL}}(\mathcal{G}', \mathcal{G}'')$ is discarded in Eq. 10.
- w/o KNNGCL: Similarly, this variant removes the GCL between random and KNN graph views by discarding the loss term $\lambda_{\text{GCL}}(\mathcal{G}'', \mathcal{G}^{\text{KNN}})$ in Eq. 10.
- w/o neighbor pos: In this variant, we remove the neighbor-based positive sample selection strategy. As a result, positive pairs can only be the same nodes in different graph views.

We observe the following according to Tables 3 and 4: (1) all components contribute to the final performance of GCPAL. For example, the variants w/o randGCL^{LF} and w/o KNNGCL^{LF} yield F1 scores of 75.9% and 76.8%, respectively, whereas the entire KGCPAL produces the highest 78.9%. (2) Among the three variants, w/o randGCL^{LF} shows the least performance degradation. It yields the second-best F1 score (75.9%), only slightly worse than GCPAL^{LF}. This may be because the KNNGCL component still benefits from the random graph view in its GCL process, mitigating the loss from omitting randGCL. (3) The variant w/o neighbor pos fails to outperform the other variants, supporting the effectiveness of our neighbor-based positive sample selection strategy. This strategy helps to eliminate semantically similar samples from the negative set, addressing the pseudo-negative sample issue.

5.3.2 Analysis of Different GNNs on GCPAL

In the GCPAL model, we have chosen GIN as the base graph encoder due to its strong graph modeling capabilities. However, the performance of GCPAL with other commonly used GNN models warrants further exploration. To investigate this, we integrate three different GNN architectures (GCN, GAT, and GraphSAGE) into the GCPAL framework and evaluate their impact. The results, presented in Tables 5 and 6, demonstrate that the choice of graph encoder architecture significantly affects GCPAL's performance. Among the three alternative architectures, GraphSAGE consistently achieves the highest scores, followed closely by GAT and GCN. How-

Table 1 Anti-money laundering detection results of GCPAL and baseline methods on the Elliptic dataset with different training ratios. We use bold fonts to highlight the best results

Method	Illicit (1%)			Illicit (2%)			Illicit (5%)		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
GCN ^{LF}	0.517 $\pm$ 0.184	0.318 $\pm$ 0.130	0.382 $\pm$ 0.141	0.767 $\pm$ 0.041	0.358 $\pm$ 0.077	0.483 $\pm$ 0.081	0.800 $\pm$ 0.026	0.414 $\pm$ 0.027	0.545 $\pm$ 0.022
GCN ^{AF}	0.441 $\pm$ 0.111	0.344 $\pm$ 0.078	0.380 $\pm$ 0.080	0.491 $\pm$ 0.156	0.261 $\pm$ 0.118	0.329 $\pm$ 0.139	0.784 $\pm$ 0.018	0.412 $\pm$ 0.014	0.540 $\pm$ 0.016
GAT ^{LF}	0.481 $\pm$ 0.355	0.221 $\pm$ 0.179	0.292 $\pm$ 0.223	0.555 $\pm$ 0.279	0.415 $\pm$ 0.215	0.473 $\pm$ 0.239	0.728 $\pm$ 0.015	0.538 $\pm$ 0.030	0.619 $\pm$ 0.023
GAT ^{AF}	0.459 $\pm$ 0.174	0.243 $\pm$ 0.161	0.307 $\pm$ 0.168	0.541 $\pm$ 0.111	0.307 $\pm$ 0.139	0.383 $\pm$ 0.149	0.712 $\pm$ 0.044	0.486 $\pm$ 0.048	0.576 $\pm$ 0.036
GraphSAGE ^{LF}	0.425 $\pm$ 0.309	0.176 $\pm$ 0.213	0.225 $\pm$ 0.271	0.608 $\pm$ 0.306	0.381 $\pm$ 0.192	0.468 $\pm$ 0.236	0.819 $\pm$ 0.021	0.592 $\pm$ 0.052	0.686 $\pm$ 0.032
GraphSAGE ^{AF}	0.682 $\pm$ 0.127	0.211 $\pm$ 0.076	0.309 $\pm$ 0.066	0.671 $\pm$ 0.194	0.343 $\pm$ 0.159	0.447 $\pm$ 0.191	0.812 $\pm$ 0.022	0.613 $\pm$ 0.024	0.699 $\pm$ 0.023
GIN ^{LF}	0.163 $\pm$ 0.327	0.086 $\pm$ 0.172	0.113 $\pm$ 0.226	0.425 $\pm$ 0.316	0.503 $\pm$ 0.320	0.377 $\pm$ 0.243	0.144 $\pm$ 0.289	0.104 $\pm$ 0.209	0.121 $\pm$ 0.243
GIN ^{AF}	0.294 $\pm$ 0.247	0.467 $\pm$ 0.327	0.281 $\pm$ 0.183	0.160 $\pm$ 0.252	0.086 $\pm$ 0.172	0.104 $\pm$ 0.207	0.404 $\pm$ 0.338	0.311 $\pm$ 0.257	0.348 $\pm$ 0.286
Skip-GCN ^{AF}	0.400 $\pm$ 0.045	0.329 $\pm$ 0.041	0.358 $\pm$ 0.026	0.626 $\pm$ 0.116	0.357 $\pm$ 0.061	0.453 $\pm$ 0.075	0.788 $\pm$ 0.025	0.419 $\pm$ 0.017	0.547 $\pm$ 0.010
Evolve-GCN ^{AF}	0.417 $\pm$ 0.032	0.331 $\pm$ 0.062	0.369 $\pm$ 0.041	0.661 $\pm$ 0.074	0.356 $\pm$ 0.024	0.463 $\pm$ 0.056	0.810 $\pm$ 0.033	0.451 $\pm$ 0.062	0.579 $\pm$ 0.075
OCGTL ^{LF}	0.545 $\pm$ 0.044	0.509 $\pm$ 0.031	0.536 $\pm$ 0.039	0.762 $\pm$ 0.031	0.529 $\pm$ 0.056	0.591 $\pm$ 0.035	0.828 $\pm$ 0.025	0.755 $\pm$ 0.020	0.798 $\pm$ 0.018
OCGTL ^{AF}	0.609 $\pm$ 0.074	0.522 $\pm$ 0.051	0.621 $\pm$ 0.063	0.738 $\pm$ 0.036	0.615 $\pm$ 0.031	0.649 $\pm$ 0.023	0.831 $\pm$ 0.021	0.772 $\pm$ 0.019	0.803 $\pm$ 0.017
Logistic Regression ^{LF}	0.382 $\pm$ 0.095	0.109 $\pm$ 0.054	0.161 $\pm$ 0.066	0.466 $\pm$ 0.089	0.069 $\pm$ 0.019	0.120 $\pm$ 0.032	0.549 $\pm$ 0.079	0.085 $\pm$ 0.016	0.146 $\pm$ 0.020
Logistic Regression ^{AF}	0.569 $\pm$ 0.045	0.642 $\pm$ 0.078	0.598 $\pm$ 0.033	0.687 $\pm$ 0.025	0.639 $\pm$ 0.030	0.661 $\pm$ 0.009	0.745 $\pm$ 0.018	0.623 $\pm$ 0.039	0.678 $\pm$ 0.026
MLP ^{LF}	0.709 $\pm$ 0.050	0.725 $\pm$ 0.039	0.717 $\pm$ 0.025	0.804 $\pm$ 0.038	0.726 $\pm$ 0.023	0.763 $\pm$ 0.015	0.821 $\pm$ 0.018	0.738 $\pm$ 0.013	0.777 $\pm$ 0.011
MLP ^{AF}	0.626 $\pm$ 0.090	0.719 $\pm$ 0.059	0.669 $\pm$ 0.076	0.720 $\pm$ 0.029	0.741 $\pm$ 0.027	0.730 $\pm$ 0.028	0.818 $\pm$ 0.015	0.764 $\pm$ 0.008	0.794 $\pm$ 0.009
Inspection-L ^{LF}	0.792 $\pm$ 0.049	0.711 $\pm$ 0.037	0.749 $\pm$ 0.024	0.827 $\pm$ 0.017	0.737 $\pm$ 0.009	0.779 $\pm$ 0.010	0.833 $\pm$ 0.044	0.785 $\pm$ 0.026	0.808 $\pm$ 0.010
Inspection-L ^{AF}	0.751 $\pm$ 0.049	0.701 $\pm$ 0.028	0.723 $\pm$ 0.019	0.813 $\pm$ 0.027	0.722 $\pm$ 0.030	0.765 $\pm$ 0.015	0.841 $\pm$ 0.011	0.782 $\pm$ 0.015	0.810 $\pm$ 0.008
GAE ^{LF}	0.668 $\pm$ 0.047	0.499 $\pm$ 0.032	0.571 $\pm$ 0.035	0.672 $\pm$ 0.021	0.594 $\pm$ 0.028	0.587 $\pm$ 0.033	0.652 $\pm$ 0.015	0.655 $\pm$ 0.019	0.654 $\pm$ 0.014
GAE ^{AF}	0.660 $\pm$ 0.041	0.646 $\pm$ 0.024	0.653 $\pm$ 0.022	0.700 $\pm$ 0.020	0.711 $\pm$ 0.031	0.706 $\pm$ 0.023	0.791 $\pm$ 0.014	0.716 $\pm$ 0.016	0.751 $\pm$ 0.024
GraphMAE ^{LF}	0.817 $\pm$ 0.051	0.705 $\pm$ 0.033	0.747 $\pm$ 0.039	0.818 $\pm$ 0.024	0.753 $\pm$ 0.015	0.766 $\pm$ 0.018	0.822 $\pm$ 0.036	0.740 $\pm$ 0.022	0.796 $\pm$ 0.017
GraphMAE ^{AF}	0.758 $\pm$ 0.056	0.704 $\pm$ 0.021	0.729 $\pm$ 0.031	0.788 $\pm$ 0.027	0.758 $\pm$ 0.039	0.781 $\pm$ 0.028	0.838 $\pm$ 0.020	0.780 $\pm$ 0.011	0.805 $\pm$ 0.017
GCPAL ^{LF}	0.823$\pm$0.044	0.742$\pm$0.037	0.780$\pm$0.012	0.846 $\pm$ 0.009	0.768$\pm$0.011	0.793$\pm$0.008	0.834 $\pm$ 0.024	0.790$\pm$0.009	0.811 $\pm$ 0.010
GCPAL ^{AF}	0.765 $\pm$ 0.041	0.713 $\pm$ 0.039	0.738 $\pm$ 0.038	0.852$\pm$0.040	0.739 $\pm$ 0.023	0.791 $\pm$ 0.022	0.868$\pm$0.021	0.789 $\pm$ 0.008	0.826$\pm$0.009

Method	Illicit (10%)			Illicit (20%)		
	Precision	Recall	F1	Precision	Recall	F1
GCN ^{LF}	0.808 $\pm$ 0.025	0.423 $\pm$ 0.011	0.555 $\pm$ 0.004	0.802 $\pm$ 0.008	0.458 $\pm$ 0.009	0.583 $\pm$ 0.006
GCN ^{AF}	0.798 $\pm$ 0.013	0.442 $\pm$ 0.010	0.569 $\pm$ 0.006	0.823 $\pm$ 0.011	0.455 $\pm$ 0.025	0.545 $\pm$ 0.022
GAT ^{LF}	0.755 $\pm$ 0.030	0.618 $\pm$ 0.033	0.679 $\pm$ 0.024	0.768 $\pm$ 0.012	0.632 $\pm$ 0.048	0.692 $\pm$ 0.032
GAT ^{AF}	0.723 $\pm$ 0.028	0.600 $\pm$ 0.039	0.656 $\pm$ 0.033	0.775 $\pm$ 0.011	0.650 $\pm$ 0.037	0.707 $\pm$ 0.025
GraphSAGE ^{LF}	0.849 $\pm$ 0.008	0.687 $\pm$ 0.016	0.759 $\pm$ 0.008	0.878 $\pm$ 0.009	0.739 $\pm$ 0.012	0.802 $\pm$ 0.004
GraphSAGE ^{AF}	0.863 $\pm$ 0.023	0.687 $\pm$ 0.024	0.764 $\pm$ 0.019	0.903 $\pm$ 0.012	0.742 $\pm$ 0.019	0.814 $\pm$ 0.010
GIN ^{LF}	0.455 $\pm$ 0.375	0.305 $\pm$ 0.283	0.349 $\pm$ 0.301	0.456 $\pm$ 0.373	0.384 $\pm$ 0.315	0.415 $\pm$ 0.339
GIN ^{AF}	0.488 $\pm$ 0.319	0.696 $\pm$ 0.294	0.408 $\pm$ 0.226	0.809 $\pm$ 0.033	0.618 $\pm$ 0.096	0.695 $\pm$ 0.062
Skip-GCN ^{AF}	0.811 $\pm$ 0.012	0.436 $\pm$ 0.011	0.567 $\pm$ 0.008	0.824 $\pm$ 0.010	0.447 $\pm$ 0.028	0.579 $\pm$ 0.024
Evolve-GCN ^{AF}	0.834 $\pm$ 0.036	0.478 $\pm$ 0.038	0.601 $\pm$ 0.043	0.845 $\pm$ 0.021	0.468 $\pm$ 0.033	0.602 $\pm$ 0.037
OCGTL ^{LF}	0.865 $\pm$ 0.025	0.751 $\pm$ 0.031	0.827 $\pm$ 0.018	0.870 $\pm$ 0.009	0.798 $\pm$ 0.014	0.839 $\pm$ 0.011
OCGTL ^{AF}	0.852 $\pm$ 0.021	0.774 $\pm$ 0.019	0.818 $\pm$ 0.015	0.862 $\pm$ 0.010	0.813 $\pm$ 0.012	0.846 $\pm$ 0.007
Logistic Regression ^{LF}	0.651 $\pm$ 0.060	0.081 $\pm$ 0.018	0.142 $\pm$ 0.029	0.704 $\pm$ 0.029	0.071 $\pm$ 0.009	0.129 $\pm$ 0.015
Logistic Regression ^{AF}	0.798 $\pm$ 0.013	0.647 $\pm$ 0.027	0.714 $\pm$ 0.012	0.803 $\pm$ 0.008	0.662 $\pm$ 0.015	0.726 $\pm$ 0.006
MLP ^{LF}	0.847 $\pm$ 0.016	0.766 $\pm$ 0.016	0.804 $\pm$ 0.006	0.878 $\pm$ 0.004	0.781 $\pm$ 0.009	0.827 $\pm$ 0.004
MLP ^{AF}	0.850 $\pm$ 0.009	0.767 $\pm$ 0.010	0.806 $\pm$ 0.004	0.884 $\pm$ 0.005	0.791 $\pm$ 0.008	0.835 $\pm$ 0.004
Inspection-L ^{LF}	0.875 $\pm$ 0.011	0.784 $\pm$ 0.007	0.827 $\pm$ 0.005	0.893 $\pm$ 0.024	0.804 $\pm$ 0.014	0.846 $\pm$ 0.009

Table 1 continued

Method	Illicit (10%)			Illicit (20%)		
	Precision	Recall	F1	Precision	Recall	F1
Inspection-L ^{AF}	<u>0.880$\pm$0.012</u>	<u>0.808$\pm$0.007</u>	<u>0.842$\pm$0.007</u>	0.877 $\pm$ 0.013	<u>0.842$\pm$0.005</u>	<u>0.859$\pm$0.007</u>
GAE ^{LF}	0.773 $\pm$ 0.027	0.620 $\pm$ 0.018	0.688 $\pm$ 0.022	0.792 $\pm$ 0.026	0.732 $\pm$ 0.029	0.693 $\pm$ 0.015
GAE ^{AF}	0.784 $\pm$ 0.021	0.773 $\pm$ 0.027	0.836 $\pm$ 0.016	0.871 $\pm$ 0.029	0.774 $\pm$ 0.011	0.819 $\pm$ 0.019
GraphMAE ^{LF}	0.842 $\pm$ 0.019	0.780 $\pm$ 0.013	0.810 $\pm$ 0.011	0.875 $\pm$ 0.035	0.794 $\pm$ 0.022	0.849 $\pm$ 0.017
GraphMAE ^{AF}	0.877 $\pm$ 0.016	0.799 $\pm$ 0.015	0.836 $\pm$ 0.009	0.881 $\pm$ 0.015	0.817 $\pm$ 0.009	0.846 $\pm$ 0.016
GCPAL ^{LF}	0.858 $\pm$ 0.017	0.820 $\pm$ 0.016	0.839 $\pm$ 0.005	0.887 $\pm$ 0.010	0.828 $\pm$ 0.013	0.856 $\pm$ 0.007
GCPAL ^{AF}	0.890$\pm$0.014	0.825$\pm$0.012	0.856$\pm$0.004	0.901$\pm$0.015	0.846$\pm$0.013	0.873$\pm$0.004

LF denotes the local raw features (e.g., the first 94 features), and AF denotes the all raw features. Scores in bold indicate the best results, while underlining indicates the second-best results

Table 2 Anti-money laundering detection results of GCPAL and baseline methods on the AMLworld dataset with training ratios 40% and 60%

Method	AMLworld (40%)			AMLworld (60%)		
	Precision	Recall	F1	Precision	Recall	F1
GCN	0.548 $\pm$ 0.031	0.303 $\pm$ 0.042	0.390 $\pm$ 0.042	0.613 $\pm$ 0.013	0.340 $\pm$ 0.020	0.437 $\pm$ 0.016
GAT	0.512 $\pm$ 0.012	0.421 $\pm$ 0.048	0.462 $\pm$ 0.032	0.580 $\pm$ 0.030	0.475 $\pm$ 0.033	0.522 $\pm$ 0.024
SAGE	0.585 $\pm$ 0.019	0.492 $\pm$ 0.022	0.534 $\pm$ 0.024	0.613 $\pm$ 0.018	0.528 $\pm$ 0.036	0.563 $\pm$ 0.028
GIN	0.539 $\pm$ 0.033	0.412 $\pm$ 0.046	0.467 $\pm$ 0.032	0.375 $\pm$ 0.319	0.535 $\pm$ 0.294	0.440 $\pm$ 0.226
Logistic Regr	0.535 $\pm$ 0.034	0.441 $\pm$ 0.023	0.483 $\pm$ 0.026	0.613 $\pm$ 0.043	0.497 $\pm$ 0.037	0.548 $\pm$ 0.032
MLP	0.385 $\pm$ 0.004	0.420 $\pm$ 0.009	0.392 $\pm$ 0.004	0.451 $\pm$ 0.016	0.489 $\pm$ 0.016	0.418 $\pm$ 0.011
SkipGCN	0.549 $\pm$ 0.010	0.298 $\pm$ 0.028	0.386 $\pm$ 0.024	0.623 $\pm$ 0.017	0.335 $\pm$ 0.021	0.435 $\pm$ 0.017
OCGTL	0.545 $\pm$ 0.027	0.518 $\pm$ 0.034	0.539 $\pm$ 0.018	0.632 $\pm$ 0.021	0.541 $\pm$ 0.027	<u>0.626$\pm$0.024</u>
GAE	0.527 $\pm$ 0.016	0.334 $\pm$ 0.031	0.419 $\pm$ 0.029	0.618 $\pm$ 0.026	0.422 $\pm$ 0.018	0.530 $\pm$ 0.022
GraphMAE	0.553 $\pm$ 0.015	<u>0.545$\pm$0.022</u>	0.531 $\pm$ 0.031	0.619 $\pm$ 0.023	0.566 $\pm$ 0.018	0.597 $\pm$ 0.014
Inspection-L	<u>0.565$\pm$0.014</u>	0.536 $\pm$ 0.029	<u>0.550$\pm$0.019</u>	<u>0.643$\pm$0.019</u>	<u>0.583$\pm$0.017</u>	0.612 $\pm$ 0.025
GCPAL	0.600$\pm$0.020	0.564$\pm$0.015	0.581$\pm$0.014	0.684$\pm$0.017	0.634$\pm$0.009	0.658$\pm$0.011

Table 3 Ablation study of GCPAL on the Elliptic dataset. We use bold fonts to highlight the best results

Method	Illicit (1%)		
	Precision	Recall	F1
w/o randGCL ^{LF}	<u>0.845$\pm$0.028</u>	0.691 $\pm$ 0.048	<u>0.759$\pm$0.033</u>
w/o randGCL ^{AF}	0.784 $\pm$ 0.067	<u>0.722$\pm$0.034</u>	0.749 $\pm$ 0.027
w/o KNNGCL ^{LF}	0.823 $\pm$ 0.022	0.720 $\pm$ 0.017	0.768 $\pm$ 0.018
w/o KNNGCL ^{AF}	0.803 $\pm$ 0.033	0.693 $\pm$ 0.026	0.743 $\pm$ 0.019
w/o neighbor pos ^{LF}	0.801 $\pm$ 0.051	0.716 $\pm$ 0.044	0.756 $\pm$ 0.025
w/o neighbor pos ^{AF}	0.718 $\pm$ 0.036	0.704 $\pm$ 0.052	0.711 $\pm$ 0.038
GCPAL ^{LF}	0.823$\pm$0.044	0.742$\pm$0.037	0.780$\pm$0.012
GCPAL ^{AF}	0.765 $\pm$ 0.041	0.713 $\pm$ 0.039	0.738 $\pm$ 0.038

Table 4 Ablation study of GCPAL on the AMLworld dataset. We use bold fonts to highlight the best results

Method	AMLworld (40%)		
	Precision	Recall	F1
w/o randGCL	0.572 $\pm$ 0.022	0.536 $\pm$ 0.028	0.536 $\pm$ 0.035
w/o KNNGCL	0.583 $\pm$ 0.031	0.538 $\pm$ 0.022	0.538 $\pm$ 0.016
w/o neighbor pos	0.547 $\pm$ 0.036	0.510 $\pm$ 0.047	0.510 $\pm$ 0.032
GCPAL	0.600$\pm$0.020	0.564$\pm$0.015	0.581$\pm$0.014

their ability to capture complex relationships among nodes in the graph.

5.3.3 Analysis of Different Feature Subsets for KNN Graph Construction

ever, the GCPAL framework with the GIN encoder yields the best performance overall. This highlights the importance of selecting the most suitable graph neural network architecture for graph encoding, as different models may vary in

We have investigated the influence of using different feature subsets (LF and AF) for KNN graph construction on the Elliptic dataset. The results are presented in Table 7. We can observe that as the features used for AML model

Table 5 The performances of different GNNs on the Elliptic dataset. We use bold fonts to highlight the best results

Method	Illicit (1%)		
	Precision	Recall	F1
w/ GCN ^{LF}	0.642 $\pm$ 0.015	0.525 $\pm$ 0.033	0.577 $\pm$ 0.017
w/ GCN ^{AF}	0.689 $\pm$ 0.043	0.531 $\pm$ 0.038	0.598 $\pm$ 0.022
w/ GAT ^{LF}	0.738 $\pm$ 0.034	0.595 $\pm$ 0.022	0.658 $\pm$ 0.016
w/ GAT ^{AF}	0.744 $\pm$ 0.027	0.612 $\pm$ 0.030	0.671 $\pm$ 0.026
w/ GraphSAGE ^{LF}	0.797 $\pm$ 0.028	0.710 $\pm$ 0.018	0.751 $\pm$ 0.016
w/ GraphSAGE ^{AF}	0.781 $\pm$ 0.034	0.723 $\pm$ 0.009	0.750 $\pm$ 0.019
GCPAL ^{LF}	0.823$\pm$0.044	0.742$\pm$0.037	0.780$\pm$0.012
GCPAL ^{AF}	0.765 $\pm$ 0.041	0.713 $\pm$ 0.039	0.738 $\pm$ 0.038

Table 6 The performances of different GNNs on the AMLworld dataset. We use bold fonts to highlight the best results

Method	AMLworld (40%)		
	Precision	Recall	F1
w/ GCL	0.548 $\pm$ 0.034	0.515 $\pm$ 0.022	0.531 $\pm$ 0.016
w/ GAT	0.564 $\pm$ 0.027	0.532 $\pm$ 0.030	0.548 $\pm$ 0.026
w/ SAGE	0.587 $\pm$ 0.028	0.540 $\pm$ 0.018	0.563 $\pm$ 0.016
GCPAL	0.600$\pm$0.020	0.564$\pm$0.015	0.581$\pm$0.014

training remain constant, the impact of varying KNN graph construction appears to be minor. For example, when the training ratio is 1% and AML training feature is LF, the F1 scores achieved by KNN-LF and KNN-AF are 0.780 $\pm$ 0.012 and 0.779 $\pm$ 0.022, respectively. This minor performance difference is consistent across all training ratios. We guess the reasons may be twofold: (1) The LF feature is sufficiently expressive to capture the essential correlations between nodes, ensuring that the constructed KNN graph remains robust and minimally affected by feature variations. (2) The two randomly augmented graph views provide graph information that compensates for minor changes in the KNN graph, maintaining the overall performance consistency.

5.4 Hyper-parameter Analysis

5.4.1 Influence of Weight λ

The hyper-parameter λ and its counterpart, $1 - \lambda$, serve as weights for the two contrastive targets in Eq. 10, specifically the contrasts between random views and the contrasts between random and KNN views. Figure 2a illustrates the performance trend of GCPAL as λ varies. From the figure, we observe that the optimal value of λ is located near the middle of the curve, rather than at the extremes (0 or 1). This suggests that both contrastive targets contribute positively to model training, with the best performance achieved when

both targets are utilized. Additionally, GCPAL performs best when λ is set to a smaller value (e.g., 0.1 for Elliptic and 0.3 for AMLworld). This indicates that contrasts between random and KNN views have a more significant impact on performance compared to the random view contrasts alone. We hypothesize that the KNN view, which relies on node correlations through raw features, better captures the similarities between illicit transactions.

5.4.2 Influence of Temperature Factor

In contrastive learning, the temperature factor τ controls the sharpness of the softmax distribution, which in turn affects the decision boundary between positive and negative samples. Figure 2b shows a bell-curve relationship between τ and performance, with the peak observed at $\tau = 0.5$ on the AMLworld dataset before the performance begins to decline. This suggests that a moderate temperature value strikes an optimal balance, allowing the model to focus on informative positive pairs while also exploring a diverse range of negative samples, thereby enhancing representation learning. On the other hand, an excessively high value or low value of τ disrupts this balance, either diminishing the model's ability to discriminate between positive and negative samples or hindering convergence. This emphasizes the importance of carefully tuning τ for achieving optimal performance.

5.4.3 Influence of Neighbor Number k

The number of neighbors, k , used to construct the KNN graph plays a crucial role in the performance of the GCPAL model. As shown in Fig. 2c, when k is small (e.g., 1 or 3), the F1 score is low, suggesting that the model is unable to effectively leverage the KNN view. As k increases, the F1 score generally improves, reaching its peak at $k = 15$, before declining. This indicates that a moderate number of neighbors provides the most valuable information for the model. When k is too small, the model struggles to capture meaningful relationships between nodes, leading to suboptimal performance. Conversely, a very large k introduces excess noise from irrelevant neighbors, which can hinder the model's ability to distinguish between legal and illegal transactions. Therefore, selecting an appropriate k is critical for optimizing performance and capturing the underlying patterns of illicit transactions.

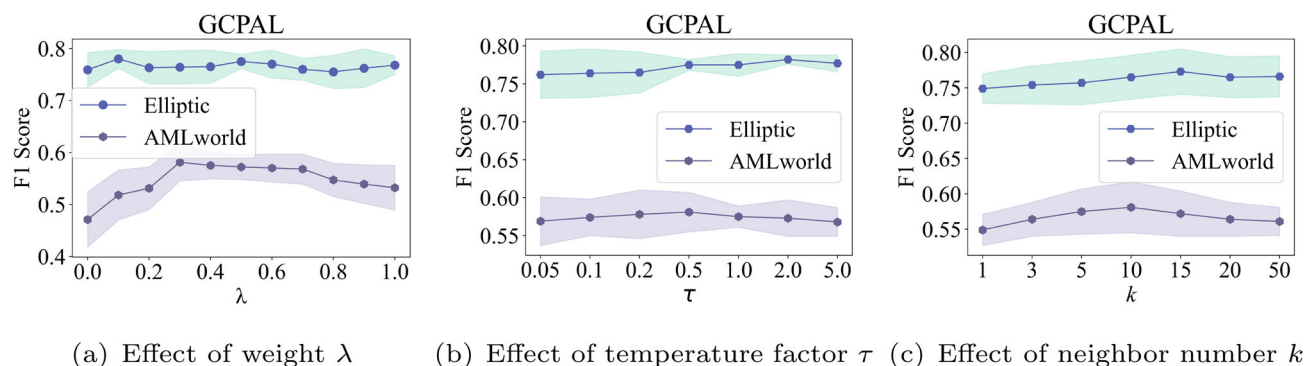
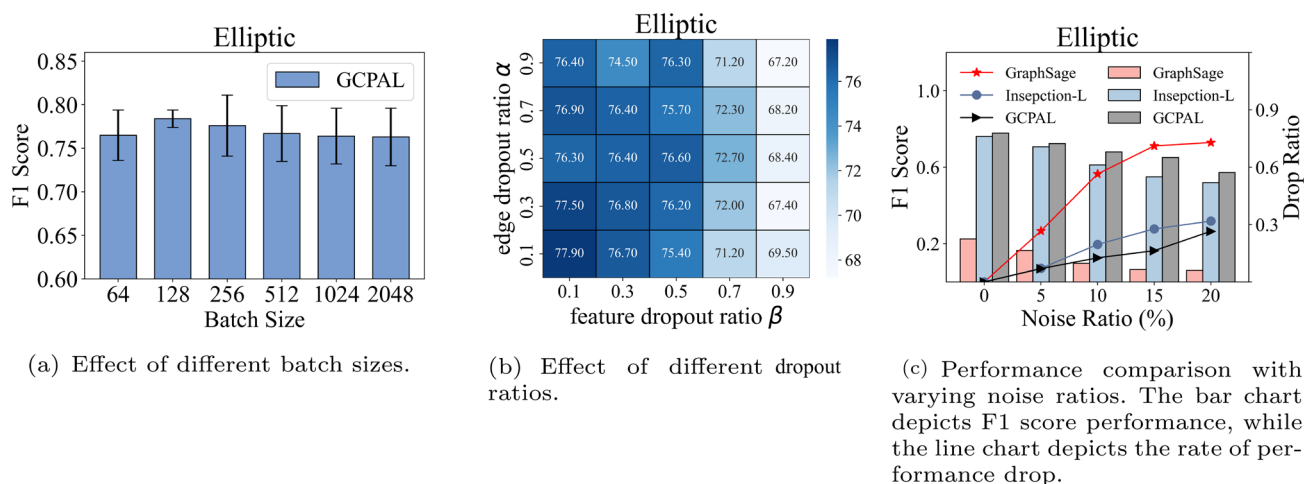
5.4.4 Influence of Batch Size

We examine the impact of batch size on the training of the GCPAL model. As shown in Fig. 3a, smaller batch sizes tend to yield slightly higher F1 scores compared to larger ones. However, the difference between the results from varying batch sizes is minimal. This suggests that the GCPAL model

Table 7 AML detection results using different features on the Elliptic dataset

AMLFeat	KNN	F1 (1%)	F1 (2%)	F1 (5%)	F1 (10%)	F1 (20%)
LF	LF	0.780 $\pm$ 0.012	0.793 $\pm$ 0.008	0.811 $\pm$ 0.010	0.839 $\pm$ 0.005	0.856 $\pm$ 0.007
	AF	0.779 $\pm$ 0.022	0.791 $\pm$ 0.011	0.813 $\pm$ 0.009	0.842 $\pm$ 0.007	0.857 $\pm$ 0.006
AF	LF	0.739 $\pm$ 0.041	0.793 $\pm$ 0.018	0.824 $\pm$ 0.013	0.852 $\pm$ 0.007	0.869 $\pm$ 0.010
	AF	0.738 $\pm$ 0.038	0.791 $\pm$ 0.022	0.826 $\pm$ 0.009	0.856 $\pm$ 0.004	0.873 $\pm$ 0.004

AMLFeat and KNN denote which features are used for AML model training and KNN graph construction, respectively. LF denotes the local raw features (the first 94 raw features), and AF denotes all features (with 72 aggregated features)

**Fig. 2** Performance comparison of SGCP w.r.t different loss weight λ , temperature factor τ , and neighbor number k **Fig. 3** Performance comparison and robust analysis of GCPAL

is relatively insensitive to changes in batch size, demonstrating consistent performance across a broad range of batch sizes. This stability may be due to the model's ability to effectively incorporate information from both the KNN graph and the random edge/feature drop perspectives, which helps mitigate the influence of batch size variations.

5.4.5 Influence of Dropout Ratio α and β

The performance of the GCPAL model is influenced by the dropout ratios α and β , which are used to create graph views through edge and feature dropout. Figure 3b demonstrates

how the F1 score changes with different combinations of dropout ratios. The results show that when the feature dropout rate is low (e.g., 0.1, 0.3, or 0.5), GCPAL achieves high F1 scores. However, when the feature dropout rate exceeds 0.5, the model's performance degrades significantly. In contrast, when the feature dropout rate is low, a higher edge dropout rate still yields good performance. This indicates that features play a more crucial role in detecting illegal transactions, necessitating a low feature dropout rate. Additionally, this supports our previous observation that a large number of edges in the transaction network may be redundant for ille-

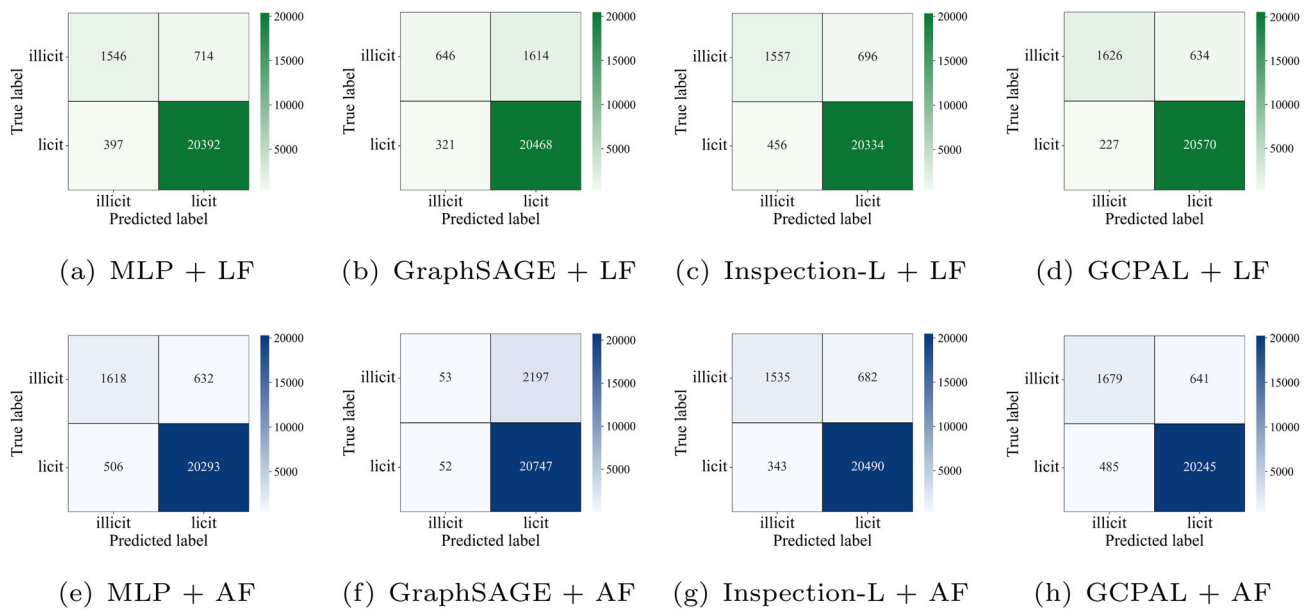


Fig. 4 Confusion matrices of different methods under the 1% training ratio

gal transaction detection, thus allowing a higher edge dropout rate.

5.4.6 Robust Analysis

We also conduct experiments on the Elliptic dataset to evaluate GCPAL's robustness against noisy features and edges. Specifically, we randomly add noise edges (e.g., by connecting nodes at random) and noise features (e.g., by replacing some feature values with Gaussian noise) at varying proportions (5%, 10%, 15%, and 20%) to the clean data. Figure 3c shows that adding noise reduces the performance of all three methods. However, GCPAL consistently exhibits a lower performance drop compared to the others, and even with a 20% noise ratio, it outperforms Inspection-L. This is likely due to GCPAL's use of edge and feature dropout augmentations to mitigate noise effects, as well as its inclusion of more negative samples in the graph contrastive pre-training, which enhances its discriminative power and robustness to noise perturbations.

5.4.7 Analysis of Confusion Matrix

Confusion matrices provide valuable insights into the performance of various classification models. In this section, we present the confusion matrices for four methods with a training ratio of 1%. Ideally, a perfect classifier would correctly classify all instances, placing them along the matrix's diagonal. As shown in Fig. 4, GCPAL consistently identifies the most illegal transactions accurately, whether using LF or AF features, while GraphSAGE performs the worst due to

its limitations with small graphs. These results highlight the effectiveness of GCPAL in detecting illicit transactions by learning more accurate node representations.

5.5 Discussion for Real-World Applications

Although our GCPAL model has been validated on the Elliptic and AMLworld datasets, several challenges remain for real-world deployment. (1) Scalability for large-scale transaction networks: As the number of nodes and edges increases, memory costs and training time rise significantly. To address these challenges, we propose integrating subgraph sampling and graph condensation techniques. Subgraph sampling [59, 60] enables training on representative portions of the network, reducing computational overhead while retaining essential structural information. Similarly, graph condensation methods [61, 62], such as sparsification or dimensionality reduction, simplify the graph structure and reduce memory usage without losing critical relationships. These techniques enhance scalability, enabling efficient application of our model to large networks. (2) Data privacy concerns: to ensure data privacy in real-world scenarios, we can incorporate federated learning technology. Federated learning [63, 64] allows training across decentralized devices or servers, where data remain local and only model updates are shared, thus preserving data confidentiality while improving model performance collaboratively. Furthermore, while our model is tailored for AML, it can be adapted to other domains such as rumor detection [65, 66], malware detection [67, 68], and fraud detection [69, 70]. The data formats across these applications are largely similar, so minimal preprocess-

ing is required. In many anomaly detection tasks, labeled data are scarce, but data augmentation and contrastive learning can help reduce the dependency on labeled data. With minor adjustments, our GCPAL framework can be applied to various domains.

6 Conclusion and Future Work

This paper proposes a novel graph contrastive learning framework for AML. We leverage contrastive learning to improve the model's expressiveness from multiple augmented views. Extensive experiments and in-depth analysis demonstrate that GCPAL outperforms SOTA AML baselines, especially with scarce labeled data (e.g., 1% and 2% of the training data). In the future, we plan to focus on the development of learnable augmentations to minimize the distortion of the original semantics caused by random perturbations. In addition, fraud nodes in the networks are usually surrounded by normal nodes that have been cheated. This inherently heterophilic structure plays an important role in AML. Therefore, the problem of heterophily in transaction networks is also a research priority.

Author Contributions Conceptualization: Hanbin Lu; methodology: Hanbin Lu; formal analysis and investigation: Haosen Wang; writing—original draft preparation: Haosen Wang; writing—review and editing: Haosen Wang.

Funding This paper has not received any form of financial support. All required funding is provided by the individual investigator or his/her affiliated organization. The design, conduct, data analysis, and conclusion writing of this study were not influenced by any financial sponsor.

Data Availability Statement The data are openly available at <https://github.com/git-disl/EllipticPlusPlus> and <https://github.com/IBM/Multi-GNN>.

Conflict of interest No potential conflict of interest was reported by the author(s).

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Li, X., Li, Y., Mo, X., Xiao, H., Shen, Y., Chen, L.: Diga: guided diffusion model for graph recovery in anti-money laundering. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, August 6–10, 2023, pp. 4404–4413. ACM. <https://doi.org/10.1145/3580305.3599806>
2. Chen, T., Kornblith, S., Norouzi, M., Hinton, G.E.: A simple framework for contrastive learning of visual representations. In: Proceedings of the 37th International Conference on Machine Learning, ICML 2020, vol. 119, pp. 1597–1607
3. Chen, X., Fan, H., Girshick, R.B., He, K.: Improved baselines with momentum contrastive learning. arXiv (2020)
4. Bachman, P., Hjelm, R.D., Buchwalter, W.: Learning representations by maximizing mutual information across views. In: Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8–14, 2019, Vancouver, pp. 15509–15519 (2019). <https://proceedings.neurips.cc/paper/2019/hash/ddf354219aac374fd40b7e760ee5bb7-Abstract.html>
5. Wei, J.W., Zou, K.: EDA: easy data augmentation techniques for boosting performance on text classification tasks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, November 3–7, 2019, pp. 6381–6387. Association for Computational Linguistics (2019)
6. Giorgi, J.M., Nitski, O., Wang, B., Bader, G.D.: Declutr: deep contrastive learning for unsupervised textual representations. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, pp. 879–895. Association for Computational Linguistics (2021). <https://doi.org/10.18653/v1/2021.acl-long.72>
7. Xie, X., Sun, F., Liu, Z., Gao, J., Ding, B., Cui, B.: Contrastive pre-training for sequential recommendation (2020)
8. In, Y., Yoon, K., Park, C.: Similarity preserving adversarial graph contrastive learning. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 867–878 (2023)
9. Panigrahi, S., Sural, S., Majumdar, A.K.: Detection of intrusive activity in databases by combining multiple evidences and belief update. In: 2009 IEEE Symposium on Computational Intelligence in Cyber Security, pp. 83–90. IEEE (2009)
10. Rajput, Q., Khan, N.S., Larik, A., Haider, S.: Ontology based expert-system for suspicious transactions detection. Comput. Inf. Sci. 7(1), 103 (2014)
11. Guevara, J., Garcia-Bedoya, O., Granados, O.: Machine learning methodologies against money laundering in non-banking correspondents. In: Applied Informatics: Third International Conference, ICAI 2020, Ota, October 29–31, 2020, Proceedings 3, pp. 72–88. Springer (2020)
12. Savage, D., Wang, Q., Zhang, X., Chou, P., Yu, X.: Detection of money laundering groups: supervised learning on small networks. In: Workshops at the Thirty-First AAAI Conference on Artificial Intelligence (2017)
13. Garcia-Bedoya, O., Granados, O., Cardozo Burgos, J.: Ai against money laundering networks: the Colombian case. J. Money Laund. Control 24(1), 49–62 (2021)
14. Labib, N.M., Rizka, M.A., Shokry, A.E.M.: Survey of machine learning approaches of anti-money laundering techniques to counter terrorism finance. In: Internet of Things-Applications and Future: Proceedings of ITAF 2019, pp. 73–87. Springer (2020)

15. Khan, W., Ishrat, M.: Unveiling the depths of explainable AI: a comprehensive review. In: *Technological Advancements in Data Processing for Next Generation Intelligent Systems*, pp. 78–106 (2024)
16. Paula, E.L., Ladeira, M., Carvalho, R.N., Marzagao, T.: Deep learning anomaly detection as support fraud investigation in Brazilian exports and anti-money laundering. In: *2016 15th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pp. 954–960. IEEE (2016)
17. Han, J., Barman, U., Hayes, J., Du, J., Burgin, E., Wan, D.: Nextgen aml: distributed deep learning based language technologies to augment anti money laundering investigation. Association for Computational Linguistics (2018)
18. Khanfor, A., Nammouchi, A., Ghazzai, H., Yang, Y., Haider, M.R., Massoud, Y.: Graph neural networks-based clustering for social internet of things. In: *2020 IEEE 63rd International Midwest Symposium on Circuits and Systems (MWSCAS)*, pp. 1056–1059. IEEE (2020)
19. Wu, S., Tang, Y., Zhu, Y., Wang, L., Xie, X., Tan, T.: Session-based recommendation with graph neural networks. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, pp. 346–353 (2019)
20. Khan, W., Ishrat, M., Khan, A.N., Arif, M., Shaikh, A.A., Khubrani, M.M., Alam, S., Shuaib, M., John, R.: Detecting anomalies in attributed networks through sparse canonical correlation analysis combined with random masking and padding. *IEEE Access* (2024)
21. Khan, W., Mohd, A., Suaib, M., Ishrat, M., Shaikh, A.A., Faisal, S.M.: Residual-enhanced graph convolutional networks with hypersphere mapping for anomaly detection in attributed networks. *Data Sci. Manag.* (2024)
22. Khan, W., Abidin, S., Arif, M., Ishrat, M., Haleem, M., Shaikh, A.A., Farooqui, N.A., Faisal, S.M.: Anomalous node detection in attributed social networks using dual variational autoencoder with generative adversarial networks. *Data Sci. Manag.* **7**(2), 89–98 (2024)
23. Diehl, F., Brunner, T., Le, M.T., Knoll, A.: Graph neural networks for modelling traffic participant interaction. In: *2019 IEEE Intelligent Vehicles Symposium (IV)*, pp. 695–701. IEEE (2019)
24. Alarab, I., Prakoonwit, S., Nacer, M.I.: Competence of graph convolutional networks for anti-money laundering in bitcoin blockchain. In: *Proceedings of the 2020 5th International Conference on Machine Learning Technologies*, pp. 23–27 (2020)
25. Shi, S., Chen, J., Wang, Z., Zhang, Y., Zhang, Y., Fu, C., Qiao, K., Yan, B.: Sstackgnn: graph data augmentation simplified stacking graph neural network for twitter bot detection. *Int. J. Comput. Intell. Syst.* **17**(1), 106 (2024). <https://doi.org/10.1007/S44196-024-00496-7>
26. Pareja, A., Domeniconi, G., Chen, J., Ma, T., Suzumura, T., Kanezashi, H., Kaler, T., Schardl, T., Leiserson, C.: Evolvegcn: evolving graph convolutional networks for dynamic graphs. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 5363–5370 (2020)
27. Huang, H., Wang, P., Zhang, Z., Zhao, Q.: A spatio-temporal attention-based gcn for anti-money laundering transaction detection. In: *International Conference on Advanced Data Mining and Applications*, pp. 634–648. Springer (2023)
28. Wei, T., Zeng, B., Guo, W., Guo, Z., Tu, S., Xu, L.: A dynamic graph convolutional network for anti-money laundering. In: *International Conference on Intelligent Computing*, pp. 493–502. Springer (2023)
29. Alarab, I., Prakoonwit, S.: Graph-based lstm for anti-money laundering: Experimenting temporal graph convolutional network with bitcoin data. *Neural Process. Lett.* **55**(1), 689–707 (2023)
30. Hyun, W., Lee, J., Suh, B.: Anti-money laundering in cryptocurrency via multi-relational graph neural network. In: *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pp. 118–130. Springer (2023)
31. Fredrik Johannessen, M.J.: Finding money launderers using heterogeneous graph neural networks. arXiv
32. Humranan, P., Supratid, S.: A study on gcn using focal loss on class-imbalanced bitcoin transaction for anti-money laundering detection. In: *2023 International Electrical Engineering Congress (iEECON)*, pp. 101–104. IEEE (2023)
33. Khan, W., Haroon, M., Khan, A.N., Hasan, M.K., Khan, A., Mokhtar, U.A., Islam, S.: Dvaeagmm: dual variational autoencoder with gaussian mixture model for anomaly detection on attributed networks. *IEEE Access* **10**, 91160–91176 (2022)
34. Khan, W., Haroon, M.: An unsupervised deep learning ensemble model for anomaly detection in static attributed social networks. *Int. J. Cogn. Comput. Eng.* **3**, 153–160 (2022)
35. Khan, W., Haroon, M.: An efficient framework for anomaly detection in attributed social networks. *Int. J. Inf. Technol.* **14**(6), 3069–3076 (2022)
36. Khan, W., Haroon, M.: A pilot study and survey on methods for anomaly detection in online social networks. In: *Human-Centric Smart Computing: Proceedings of ICHCSC 2022*, pp. 119–128. Springer (2022)
37. Khan, W., Ishrat, M., Haleem, M., Khan, A.N., Hasan, M.K., Farooqui, N.A.: An extensive study and review on dark web threats and detection techniques. In: *Advances in Cyberology and the Advent of the Next-Gen Information Revolution*, pp. 202–219. IGI Global (2023)
38. Lo, W.W., Kulatilleke, G.K., Sarhan, M., Layeghy, S., Portmann, M.: Inspection-l: self-supervised gnn node embeddings for money laundering detection in bitcoin. *Appl. Intell.* **53**(16), 19406–19417 (2023)
39. Cardoso, M., Saleiro, P., Bizarro, P.: Laundrograph: self-supervised graph representation learning for anti-money laundering. In: *Proceedings of the Third ACM International Conference on AI in Finance*, pp. 130–138 (2022)
40. Velickovic, P., Fedus, W., Hamilton, W.L., Liò, P., Bengio, Y., Hjelm, R.D.: Deep graph infomax. In: *7th International Conference on Learning Representations, ICLR 2019, New Orleans, May 6–9, 2019*. OpenReview.net (2019). <https://openreview.net/forum?id=rklz9iAcKQ>
41. Sun, F., Hoffmann, J., Verma, V., Tang, J.: Infograph: unsupervised and semi-supervised graph-level representation learning via mutual information maximization. In: *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, April 26–30, 2020*. OpenReview.net (2020). <https://openreview.net/forum?id=r1lff2NYvH>
42. Hassani, K., Ahmadi, A.H.K.: Contrastive multi-view representation learning on graphs. In: *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13–18 July 2020, Virtual Event. Proceedings of Machine Learning Research*, vol. 119, pp. 4116–4126. PMLR. <http://proceedings.mlr.press/v119/hassani20a.html>
43. Sun, Q., Li, J., Peng, H., Wu, J., Ning, Y., Yu, P.S., He, L.: SUGAR: subgraph neural network with reinforcement pooling and self-supervised mutual information mechanism. In: *WWW '21: The Web Conference 2021, Virtual Event/Ljubljana, April 19–23, 2021*, pp. 2081–2091. ACM/IW3C2. <https://doi.org/10.1145/3442381.3449822>
44. Qiu, J., Chen, Q., Dong, Y., Zhang, J., Yang, H., Ding, M., Wang, K., Tang, J.: GCC: graph contrastive coding for graph neural network pre-training. In: *KDD '20: The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, August 23–27, 2020*, pp. 1150–1160. ACM (2020). <https://doi.org/10.1145/3394486.3403168>
45. Zhu, Y., Xu, Y., Yu, F., Liu, Q., Wu, S., Wang, L.: Graph contrastive learning with adaptive augmentation. In: *WWW '21:*

- The Web Conference 2021, Virtual Event/Ljubljana, April 19–23, 2021, pp. 2069–2080. ACM/IW3C2 (2021). <https://doi.org/10.1145/3442381.3449802>
46. Chu, G., Wang, X., Shi, C., Jiang, X.: Cuco: graph representation with curriculum contrastive learning. In: IJCAI, pp. 2300–2306 (2021)
 47. Thakoor, S., Tallec, C., Azar, M.G., Munos, R., Veličković, P., Valko, M.: Bootstrapped representation learning on graphs. In: ICLR 2021 Workshop on Geometrical and Topological Representation Learning (2021)
 48. Weber, M., Domeniconi, G., Chen, J., Weidele, D.K.I., Bellei, C., Robinson, T., Leiserson, C.E.: Anti-money laundering in bitcoin: experimenting with graph convolutional networks for financial forensics (2019). [arXiv:1908.02591](https://arxiv.org/abs/1908.02591)
 49. Xu, K., Hu, W., Leskovec, J., Jegelka, S.: How powerful are graph neural networks? In: 7th International Conference on Learning Representations, ICLR 2019, New Orleans, May 6–9, 2019 (2019)
 50. Oord, A., Li, Y., Vinyals, O.: Representation learning with contrastive predictive coding (2018). [arXiv:1807.03748](https://arxiv.org/abs/1807.03748)
 51. Oord, A.v.d., Li, Y., Vinyals, O.: Representation learning with contrastive predictive coding (2018). [arXiv:1807.03748](https://arxiv.org/abs/1807.03748)
 52. Altman, E., Blanuša, J., Von Niederhäusern, L., Egressy, B., Anghel, A., Atasu, K.: Realistic synthetic financial transactions for anti-money laundering models. In: Advances in Neural Information Processing Systems, vol. 36 (2024)
 53. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: 5th International Conference on Learning Representations, ICLR 2017, Toulon, April 24–26, 2017, Conference Track Proceedings (2017)
 54. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y.: Graph attention networks. In: 6th International Conference on Learning Representations, ICLR 2018 (2017)
 55. Hamilton, W., Ying, Z., Leskovec, J.: Inductive Representation Learning on Large Graphs, vol. 30 (2017)
 56. Qiu, C., Kloft, M., Mandt, S., Rudolph, M.: Raising the bar in graph-level anomaly detection. In: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22, pp. 2196–2203 (2022)
 57. Kipf, T.N., Welling, M.: Variational graph auto-encoders (2016). [arXiv:1611.07308](https://arxiv.org/abs/1611.07308)
 58. Hou, Z., Liu, X., Cen, Y., Dong, Y., Yang, H., Wang, C., Tang, J.: Graphmae: self-supervised masked graph autoencoders. In: KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, August 14–18, 2022, pp. 594–604 (2022)
 59. Zhang, Q., Sun, Y., Hu, Y., Wang, S., Yin, B.: A subgraph sampling method for training large-scale graph convolutional network. Inf. Sci. **649**, 119661 (2023)
 60. Shu, D.W., Kim, Y., Kwon, J.: Localized curvature-based combinatorial subgraph sampling for large-scale graphs. Pattern Recogn. **139**, 109475 (2023)
 61. Gao, X., Yu, J., Chen, T., Ye, G., Zhang, W., Yin, H.: Graph condensation: a survey (2024). [arXiv:2401.11720](https://arxiv.org/abs/2401.11720)
 62. Zheng, X., Zhang, M., Chen, C., Nguyen, Q.V.H., Zhu, X., Pan, S.: Structure-free graph condensation: from large-scale graphs to condensed graph-free data. In: Advances in Neural Information Processing Systems, vol. 36 (2024)
 63. Wen, J., Zhang, Z., Lan, Y., Cui, Z., Cai, J., Zhang, W.: A survey on federated learning: challenges and applications. Int. J. Mach. Learn. Cybern. **14**(2), 513–535 (2023)
 64. Ye, M., Fang, X., Du, B., Yuen, P.C., Tao, D.: Heterogeneous federated learning: state-of-the-art and research challenges. ACM Comput. Surv. **56**(3), 1–44 (2023)
 65. Sun, T., Qian, Z., Dong, S., Li, P., Zhu, Q.: Rumor detection on social media with graph adversarial contrastive learning. In: Proceedings of the ACM Web Conference 2022, pp. 2789–2797 (2022)
 66. Wang, H., Tang, P., Kong, H., Jin, Y., Wu, C., Zhou, L.: Dhcf: dual disentangled-view hierarchical contrastive learning for fake news detection on social media. Inf. Sci. **645**, 119323 (2023)
 67. Gao, Y., Hasegawa, H., Yamaguchi, Y., Shimada, H.: Unsupervised graph contrastive learning with data augmentation for malware classification. In: Proc. 16th International Conference on Emerging Security Information, Systems and Technologies (SECURWARE 2022), IARIA, pp. 41–47 (2022)
 68. Bilot, T., El Madhoun, N., Al Agha, K., Zouaoui, A.: A survey on malware detection with graph representation learning. ACM Comput. Surv. **56**(11), 1–36 (2024)
 69. Zhang, R., Cheng, D., Yang, J., Ouyang, Y., Wu, X., Zheng, Y., Jiang, C.: Pre-trained online contrastive learning for insurance fraud detection. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, pp. 22511–22519 (2024)
 70. Cao, J., Cui, X., Zheng, C.: Tfd-gcl: telecommunications fraud detection based on graph contrastive learning with adaptive augmentation. In: 2024 International Joint Conference on Neural Networks (IJCNN), pp. 1–7. IEEE (2024)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.